

Tarea 2:

Juan Sebastian Avendaño - 202411658

Andres Felipe Rodriguez – 202322292

Parte 1:

Montos	tiempo(ms)	Crecimiento
10	2	
15	4	2
20	15	3,75
25	71	4,73333333
30	1263	17,7887324
35	14994	11,871734

Tabla 1: los tiempos medidos en milisegundos para la ejecución del algoritmo recursivo sin uso de memoria para el problema coinchange

El tiempo no crece de manera lineal por lo que no se puede estimar cuanto crece cada vez que aumenta en 5 unidades el monto, lo que sí se puede observar en la tabla 1 es que el crecimiento es exponencial y que después de 25 tiene un salto considerablemente alto en complejidad temporal. Las mediciones con montos pequeños no son relevantes para estimar el factor, ya que al implementar un algoritmo recursivo para un monto pequeño los llamados van a ser pocos y no se va a ver afectada la complejidad. Por otro lado, con montos mas grandes van a haber muchos más llamados recursivos y la complejidad va a aumentar exponencialmente, como se ve en la tabla 1 para un monto de 35 tardo 14994 ms y muy probablemente para un monto de 40 tardara horas.

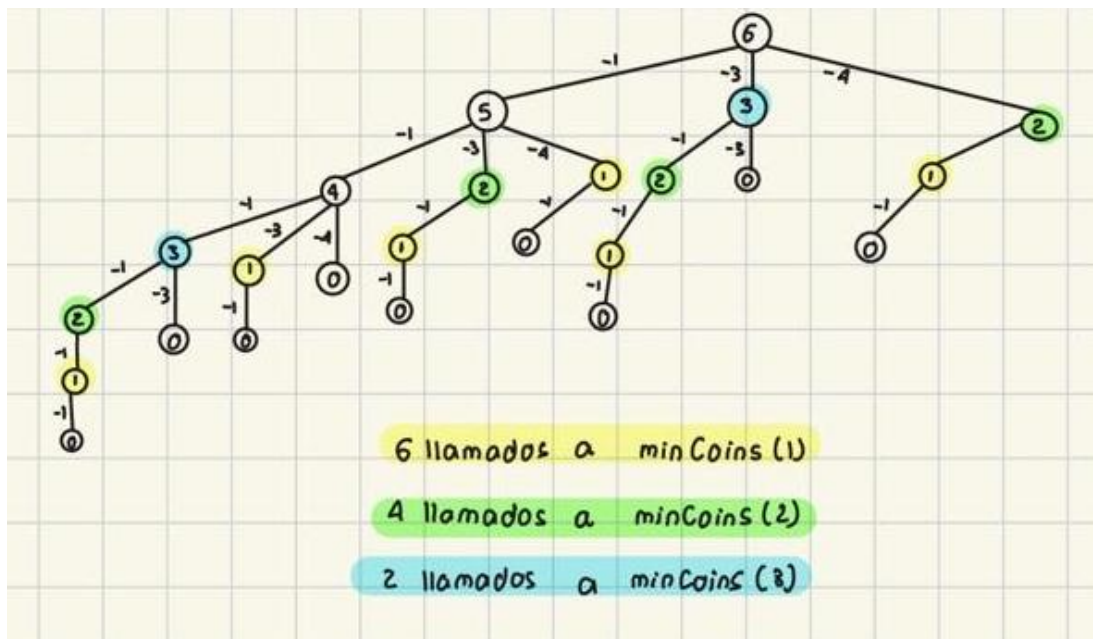


Ilustración 3: árbol de llamadas recursivos para un monto de 6, con un sistema monetario {1,3,4}

Parte 2

Memoización

Monto	Tiempo (ms)
10^2	129
10^3	143
10^4	154
10^5	275

Tabulación

Monto	Tiempo (ms)
10^2	115
10^3	120
10^4	123
10^5	143

2.1 Comparación con la complejidad teórica

Tanto Memoized como Tabular resuelven la misma recurrencia con complejidad teórica $O(n \cdot m)$ en tiempo (n denominaciones $\times$ m subproblemas) y $O(m)$ en espacio. Si la teoría se cumpliera al pie de la letra, al multiplicar m por 10 el tiempo también debería multiplicarse aproximadamente por 10, ya que n es constante. Sin embargo, los datos no muestran ese crecimiento lineal.

Esto se puede deber a que estos tamaños de datos son pequeños para una CPU moderna. De modo que lo que domina el tiempo medido no es el algoritmo sino el overhead fijo del proceso. Ese overhead es aproximadamente constante sin importar el monto, por eso los tiempos se ven casi planos hasta que m es lo bastante grande, casi 10^5 , para que el trabajo real empiece a ser una fracción no despreciable del total. En consecuencia, para ver el comportamiento $O(n \cdot m)$ de forma clara haría falta llegar a montos de 10^9 u 10^{10} , donde el trabajo algorítmico supere ampliamente el overhead de arranque.

2.2 Por qué Memoized falla con StackOverflowError y Tabular no

Memoized resuelve $C(m)$ llamando recursivamente a $C(m-d)$ para cada moneda d . Cada uno de esos llamados agrega un nuevo stack frame a la pila de ejecución de Java, y esos frames no se liberan hasta que la recursión regresa. La profundidad máxima de la pila crece proporcionalmente al monto. En el peor caso, del orden de $m / d_{\text{mín}}$, es decir, del orden de m si la moneda más pequeña es 1. Como la pila de un hilo en Java tiene un tamaño fijo por defecto, al llegar a montos grandes, se agotan los frames disponibles y ocurre StackOverflowError. Por otro lado, Tabular calcula las respuestas con un ciclo iterativo ascendente, sin ningún llamado recursivo. Esto significa que usa siempre la misma cantidad constante de stack frames sin importar qué tan grande sea el monto. Las tablas: minCoinsTable y lastCoin crecen en el heap, no en el stack, y el heap es mucho más grande y puede crecer dinámicamente, por lo que no hay riesgo de desbordamiento por este motivo.

Lo anterior se puede evitar agrandando el tamaño de la pila con `-Xss` al ejecutar la JVM por medio de powershell. Esto le da a cada hilo más espacio de stack, permitiendo mayor profundidad de recursión antes de fallar. También se puede intentar reescribir el algoritmo de modo que no se use recursión, en cambio hacer una versión iterativa, o simular la recursión con una pila explícita en el heap.

2.3 ¿Cuánta memoria adicional necesita cada uno de los cuatro algoritmos?

- Naive: no usa tablas de memoria, pero cada llamado recursivo consume un stack frame, así que usa $O(m)$ de memoria en la pila de ejecución.

- Memoized: usa $O(m)$ en el heap para el arreglo memory, más $O(m)$ en el stack por la profundidad de la recursión.
- Tabular: usa $O(m)$ en el heap para los dos arreglos (minCoinsTable y lastCoin), pero $O(1)$ en el stack, ya que es completamente iterativo.
- Greedy: no usa ninguna tabla ni arreglo dependiente de m . En cada paso solo recuerda el monto restante y la moneda más grande que quepa. Su memoria adicional es $O(1)$.

Parte 3

Ejecución con coins_canonico.txt:

```
C:\7 semestre\Dalgo\Tarea2>java -cp bin DynamicProgrammingExample data/coins_canonico.txt data/amounts.txt out_canonico.txt Tabular
Sistema monetario (10 denominaciones): [1, 2, 5, 10, 20, 50, 100, 200, 500, 1000]
Montos a pagar: 1000. Algoritmo: Tabular
Montos resueltos. Tiempo total(milisegundos): 26
El algoritmo voraz entrego una respuesta diferente en 0 de los 1000 montos
```

Ilustración 3: ejecución con coins_canonico.txt

En este caso no fallo en ninguno, por lo que podemos observar que para este sistema monetario greedy es eficaz, esto se debe a que la diferencia entre cada digito es lo suficientemente grande para que el algoritmo funcione correctamente.

Ejecucion con sistema Colombiano:

```
C:\7 semestre\Dalgo\Tarea2>java -cp bin DynamicProgrammingExample data/coins_colombia.txt data/amounts_cop.txt out_colombia.txt Tabular
Sistema monetario (11 denominaciones): [50, 100, 200, 500, 1000, 2000, 5000, 10000, 20000, 50000, 100000]
Montos a pagar: 1000. Algoritmo: Tabular
Montos resueltos. Tiempo total(milisegundos): 21
El algoritmo voraz entrego una respuesta diferente en 0 de los 1000 montos
```

Ilustración 4: ejecución con sistema Colombiano.txt

Para este Sistema no hubo ningún fallo, se puede ver que la diferencia entre cada digito del sistema es lo suficientemente grande. Es similar al canónico pero en este caso particular es aún mejor ya que todas las denominaciones del sistema Colombiano son múltiplos de 50 por lo que sus diferencias son aun mas grandes, beneficiando así al algoritmo.

Ejecucion con sistema no Canonico:

```
C:\7 semestre\Dalgo\Tarea2>java -cp bin DynamicProgrammingExample data/coins_nocanonico.txt data/amounts.txt out_nocanonico.txt Tabular
Sistema monetario (3 denominaciones): [1, 7, 10]
Montos a pagar: 1000. Algoritmo: Tabular
Montos resueltos. Tiempo total(milisegundos): 87
Diferencia con el algoritmo voraz en el monto 37356. Tabular: 3738 monedas. Greedy: 3741 monedas
Diferencia con el algoritmo voraz en el monto 63624. Tabular: 6363 monedas. Greedy: 6366 monedas
Diferencia con el algoritmo voraz en el monto 65945. Tabular: 6596 monedas. Greedy: 6599 monedas
Diferencia con el algoritmo voraz en el monto 69435. Tabular: 6945 monedas. Greedy: 6948 monedas
Diferencia con el algoritmo voraz en el monto 63376. Tabular: 6340 monedas. Greedy: 6343 monedas
El algoritmo voraz entrego una respuesta diferente en 294 de los 1000 montos
```

Ilustración 5: ejecución con sistema no Canonico.txt

Con este sistema se ve que hay varios fallos, ya que greedy fue ineficiente en 294 de los 1000 casos. Esto se debe a que las denominaciones están demasiado cercanas entre ellas, por ende el algoritmo queda “atrapado” en algunos casos después de seleccionar el mas grande y produce una Solución no optima. Por ejemplo, para un monto de 14 tomaría uno de 10 y posteriormente tendría que completar lo que falta solo con 1, cuando la respuesta correcta seria 2 veces 7.

Ejecucion sistema McNuggets:

```

C:\7 semestre\Dalgo\Tarea2>java -cp bin DynamicProgrammingExample data/coins_mcnuggets.txt data/amounts.txt out_mcnuggets.txt Tabular
Sistema monetario (3 denominaciones): [6, 9, 20]
Montos a pagar: 1000. Algoritmo: Tabular
Montos resueltos. Tiempo total(milisegundos): 48
Diferencia con el algoritmo voraz en el monto 12307. Tabular: 617 monedas. Greedy: -1 monedas
Diferencia con el algoritmo voraz en el monto 65033. Tabular: 3254 monedas. Greedy: -1 monedas
Diferencia con el algoritmo voraz en el monto 37356. Tabular: 1870 monedas. Greedy: -1 monedas
Diferencia con el algoritmo voraz en el monto 35908. Tabular: 1799 monedas. Greedy: -1 monedas
Diferencia con el algoritmo voraz en el monto 80190. Tabular: 4012 monedas. Greedy: -1 monedas
El algoritmo voraz entrego una respuesta diferente en 729 de los 1000 montos

```

Ilustración 6: ejecución con sistema McNuggets.txt

Este sistema presenta el mismo problema que el no canónico con la diferencia que no tiene el 1, por lo que el problema es más grave. Sin 1 en el sistema monetario se producen muchas "cuentas" que son imposibles de pagar después de haber seleccionado la mayor denominación cada vez. Esto hace que Greedy no solo sea ineficiente, sino que además reporte falsamente que ciertos montos no tienen solución cuando sí la tienen, convirtiéndolo en un algoritmo completamente inadecuado para este sistema.

Para verificar cual es el monto mas pequeño con el que Greedy falla en el sistema no canónico $\{1, 7, 10\}$ usamos un archivo auxiliar .txt con montos del 1 al 15 haciendo uso de la clase AmountsGenerator:

```

C:\7 semestre\Dalgo\Tarea2>java -cp bin DynamicProgrammingExample data/coins_nocanonico.txt data/amounts_1a15.txt out_1a15.txt Tabular
Sistema monetario (3 denominaciones): [1, 7, 10]
Montos a pagar: 15. Algoritmo: Tabular
Montos resueltos. Tiempo total(milisegundos): 0
Diferencia con el algoritmo voraz en el monto 14. Tabular: 2 monedas. Greedy: 5 monedas
Diferencia con el algoritmo voraz en el monto 15. Tabular: 3 monedas. Greedy: 6 monedas
El algoritmo voraz entrego una respuesta diferente en 2 de los 15 montos

```

Ilustración 7: ejecución de verificación

En ese rango se obtuvieron 2 errores, siendo el 14 el monto mas pequeño que causa problemas.

¿por qué el algoritmo voraz es óptimo para el sistema de denominaciones colombiano?

El algoritmo voraz es óptimo para el sistema colombiano porque sus denominaciones están suficientemente espaciadas y todas son múltiplos de 50. Este espaciamiento evita que Greedy quede atrapado seleccionando la denominación mayor, garantizando que siempre lleva a la solución óptima.

¿Es suficiente con que cada denominación sea múltiplo de la anterior?

No. Ser múltiplo de la anterior no es suficiente. Contraejemplo: $\{1, 3, 4\}$ donde Greedy falla en el monto 6. Se requiere que las denominaciones estén suficientemente espaciadas para garantizar optimalidad, no solo que sean múltiplos.